

# Algorytmy i struktury danych

## Fragment rozdziału 7: tablice haszujące

Fragment testowy

2026

# Spis treści

|          |                                                |          |
|----------|------------------------------------------------|----------|
| <b>1</b> | <b>Tablice haszujące</b>                       | <b>1</b> |
| 1.1      | Zasada działania . . . . .                     | 2        |
| 1.2      | Współczynnik wypełnienia . . . . .             | 3        |
| 1.3      | Dwie strategie rozwiązywania kolizji . . . . . | 4        |
| 1.3.1    | Łańcuchowanie . . . . .                        | 4        |
| 1.3.2    | Adresowanie otwarte . . . . .                  | 5        |
| 1.4      | Powiększanie tablicy . . . . .                 | 7        |
| 1.5      | Kiedy tablica haszująca jest złym wyborem . .  | 9        |
| 1.5.1    | Ćwiczenia . . . . .                            | 10       |
| 1.6      | Słownik pojęć . . . . .                        | 11       |
| 1.7      | Uwaga o źródłach . . . . .                     | 12       |

# Rozdział 1

## Tablice haszujące

Wyszukiwanie w tablicy nieuporządkowanej kosztuje  $O(n)$ , w drzewie zrównoważonym  $O(\log n)$ . Tablica haszująca obiecuje  $O(1)$  — koszt niezależny od liczby elementów. Ten rozdział pokazuje, skąd bierze się ta obietnica, kiedy przestaje obowiązywać i co robi się, gdy przestaje. Zakładamy znajomość notacji asymptotycznej oraz podstawowych struktur listowych.

## 1.1 Zasada działania

Tablica haszująca to tablica  $T$  o rozmiarze  $m$  wraz z funkcją  $h : U \rightarrow \{0, 1, \dots, m-1\}$ , która każdemu kluczowi z uniwersum  $U$  przypisuje indeks. Element o kluczu  $k$  trafia do komórki  $T[h(k)]$ .

Cały pomysł opiera się na jednym spostrzeżeniu: jeśli miejsce elementu **wynika z jego wartości**, to nie trzeba go szukać — wystarczy policzyć, gdzie leży. Koszt wyszukiwania sprowadza się do kosztu obliczenia  $h(k)$ , a ten nie zależy od  $n$ .

Kłopot polega na tym, że  $|U|$  jest zwykle znacznie większe od  $m$ . Uniwersum napisów o długości do 20 znaków ma około  $10^{28}$  elementów; tablica ma ich może milion. Z zasady szufladkowej wynika, że muszą istnieć klucze różne, którym  $h$  przypisze ten sam indeks. Nazywamy to **kolizją** i to wokół niej kręci się cała reszta rozdziału.

## 1.2 Współczynnik wypełnienia

Kluczową wielkością opisującą stan tablicy jest **współczynnik wypełnienia**

$$\alpha = \frac{n}{m}, \quad (1.1)$$

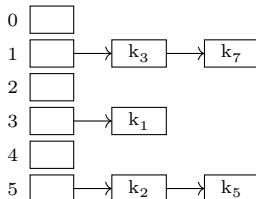
gdzie  $n$  to liczba przechowywanych elementów. Przy adresowaniu otwartym zachodzi  $\alpha \leq 1$ ; przy łańcuchowaniu  $\alpha$  może przekraczać jedność.

Wielkość (1.1) rządzi wszystkim: to od niej, a nie od  $n$ , zależy oczekiwana liczba porównań. To dlatego tablica haszująca zachowuje stały koszt przy rosnącej liczbie elementów — o ile rośnie razem z nią.

## 1.3 Dwie strategie rozwiązywania kolizji

### 1.3.1 Łańcuchowanie

Każda komórka tablicy przechowuje listę elementów o tym samym haszu. Wstawienie to dopisanie na początek listy, wyszukiwanie — przejście listy.



Rysunek 1.1: Łańcuchowanie: komórki 1 i 5 zawierają po dwa elementy o tym samym haszu. Koszt wyszukiwania rośnie z długością łańcucha, a ta zależy od współczynnika wypełnienia.

Przy założeniu haszowania równomiernego oczekiwana długość łańcucha wynosi  $\alpha$ , więc oczekiwany koszt wyszukiwania nieudanego to

$$\Theta(1 + \alpha). \quad (1.2)$$

Składnik 1 odpowiada obliczeniu funkcji haszującej, składnik  $\alpha$  — przejściu listy. Gdy  $\alpha$  jest stałe, koszt jest stały; gdy tablica przestaje rosnąć wraz z danymi,  $\alpha$  rośnie liniowo i struktura degeneruje się do listy.

### 1.3.2 Adresowanie otwarte

Wszystkie elementy leżą w samej tablicy. Przy kolizji szukamy kolejnej wolnej komórki według ustalonej sekwencji prób  $h(k, 0), h(k, 1), \dots$

Trzy najczęstsze sposoby:

Tabela 1.1: Porównanie strategii adresowania otwartego

| Metoda     | Sekwencja prób                                | Grupowanie       |
|------------|-----------------------------------------------|------------------|
| liniowa    | $h(k, i) = (h'(k) + i) \bmod m$               | pierwotne, silne |
| kwadratowa | $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$ | wtórne, słabsze  |

| Metoda    | Sekwencja prób                          | Grupowanie       |
|-----------|-----------------------------------------|------------------|
| dwukrotna | $h(k, i) = (h_1(k) + i h_2(k)) \bmod m$ | praktycznie brak |

**Grupowanie pierwotne** to zjawisko, przez które metoda liniowa działa gorzej, niż wynikałoby z rachunku: zajęte komórki układają się w ciągłe bloki, a każdy nowy element trafiający w blok wydłuża go, zwiększając prawdopodobieństwo trafienia następnego. Tab. 1.1 wymienia to w ostatniej kolumnie.

Oczekiwana liczba prób przy wyszukiwaniu nieudanym wynosi

$$\frac{1}{1 - \alpha}, \quad (1.3)$$

co ma konsekwencję praktyczną, którą trzeba znać: przy  $\alpha = 0,9$  to 10 prób, przy  $\alpha = 0,99$  — już sto. Dlatego implementacje powiększają tablicę, gdy  $\alpha$  przekroczy próg rzędu 0,7.



Tabela 1.2: Oczekiwana liczba prób w zależności od wypełnienia

| $\alpha$ | Łańcuchowanie | Adresowanie otwarte |
|----------|---------------|---------------------|
| 0,25     | 1,25          | 1,33                |
| 0,50     | 1,50          | 2,00                |
| 0,75     | 1,75          | 4,00                |
| 0,90     | 1,90          | 10,00               |
| 0,99     | 1,99          | 100,00              |

Tab. 1.2 pokazuje, dlaczego wybór strategii nie jest kwestią gustu: przy niskim wypełnieniu obie są porównywalne, przy wysokim — różnią się o dwa rzędy wielkości.

## 1.4 Powiększanie tablicy

Gdy  $\alpha$  przekroczy próg, tworzymy tablicę o rozmiarze  $2m$  i przenosimy do niej wszystkie elementy, licząc hasze na nowo. Pojedyncza taka operacja kosztuje  $O(n)$  — czyli dużo. Ale wykonuje się ją rzadko, a **koszt zamortyzowany** pozostaje stały.

Rozumowanie jest następujące. Między dwoma powiększeniami wstawiamy co najmniej  $m/2$  elementów, a samo powiększenie kosztuje  $O(m)$ . Koszt przypadający na jedno wstawienie to zatem  $O(m)/(m/2) = O(1)$ .

To jest przykład rozumowania, które warto rozpoznawać: **operacja droga, ale rzadka, może być tania w rozliczeniu na jedną operację**. Ta sama analiza stosuje się do tablic dynamicznych i do struktur zbiorów rozłącznych.

```
def wstaw(tablica, klucz, wartosc):  
    """Wstawia parę, powiększając tablicę po  
    ↪ przekroczeniu progu.  
  
    Próg 0,7 nie jest przypadkowy - patrz wzór na  
    ↪ oczekiwaną  
    liczbę prób: powyżej niego koszt rośnie gwałtownie.  
    """  
  
    if tablica.n / tablica.m > 0.7:  
        powieksz(tablica, 2 * tablica.m)  
  
    i = haszuj(klucz) % tablica.m  
    while tablica.komorki[i] is not None:
```

```

    if tablica.komorki[i].klucz == klucz:
        tablica.komorki[i].wartosc = wartosc
    return

    i = (i + 1) % tablica.m          # sondowanie
↪ liniowe

tablica.komorki[i] = Wpis(klucz, wartosc)
tablica.n += 1

```

## 1.5 Kiedy tablica haszująca jest złym wyborem

Obietnica  $O(1)$  jest **oczekiwaniem**, nie gwarancją. Cztery sytuacje, w których warto sięgnąć po co innego:

1. **Potrzebny porządek.** Tablica haszująca nie pozwala przejść elementów w kolejności rosnącej ani znaleźć następnika. Drzewo zrównoważone tak.
2. **Klucze pochodzą od przeciwnika.** Znając funkcję haszującą, można dobrać klucze tak, by wszystkie trafiły w jedną komórkę — koszt rośnie do  $O(n)$ . Obrona:

haszowanie uniwersalne z losowym dobozem funkcji przy starcie.

3. **Twarde ograniczenie czasu.** Powiększanie tablicy to jednorazowe  $O(n)$ ; w systemie czasu rzeczywistego może to być nie do przyjęcia.
4. **Bardzo mało elementów.** Przy kilkunastu wpisach przeszukanie tablicy liniowej bywa szybsze, bo mieści się w pamięci podręcznej procesora.

### 1.5.1 Ćwiczenia

1. [10] Oblicz oczekiwaną liczbę prób przy  $\alpha = 0,8$  dla obu strategii i porównaj wynik z tab. 1.2.
2. [15] Wyjaśnij, dlaczego przy adresowaniu otwartym usunięcie elementu nie może polegać na wpisaniu wartości pustej.
3. [20] Zaimplementuj sondowanie dwukrotne i sprawdź doświadczalnie, czy znosi grupowanie pierwotne.
4. [25] [M] Wyprowadź wzór (1.3), zakładając haszowanie równomierne.
5. [30] [M] Wykaż, że przy powiększaniu o stały składnik (a nie o czynnik 2) koszt zamortyzowany przestaje być stały.

6. [40] [HM] Udowodnij, że rodzina funkcji  $h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$  dla pierwszego  $p > |U|$  jest uniwersalna.

## 1.6 Słownik pojęć

**Adresowanie otwarte** — strategia rozwiązywania kolizji, w której wszystkie elementy leżą w tablicy, a przy zajętej komórce szuka się kolejnej według sekwencji prób. Wprowadzone w §1.3.

**Grupowanie pierwotne** — zjawisko zlepienia się zajętych komórek w ciągle bloki przy sondowaniu liniowym, pogarszające koszt ponad wartość wynikającą z wypełnienia. Wprowadzone w §1.3.

**Koszt zamortyzowany** — średni koszt operacji w długim ciągu operacji, przy którym pojedyncze operacje drogie rozkładają się na wiele tanich. Wprowadzony w §1.4.

**Współczynnik wypełnienia** — stosunek liczby elementów do rozmiaru tablicy,  $\alpha = n/m$ ; wielkość, od której zależy koszt wszystkich operacji. Wprowadzony w §1.2.

## 1.7 Uwaga o źródłach

Wzory na oczekiwaną liczbę prób są standardowe i można je znaleźć w każdym podręczniku algorytmów — analiza tablic haszujących pochodzi w tej postaci od Knutha, który omawia ją w trzecim tomie *The Art of Computer Programming*. Nie podaję numeru strony ani wydania, bo nie mam ich pod ręką; to fragment testowy, nie praca wymagająca aparatu bibliograficznego.

Wartości w tab. 1.2 wyliczono ze wzorów (1.2) i (1.3).